

Reviewer Pack — Prime Density in Arithmetic Progressions and Seed Length of ...

Entry ffe6d4da52cf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 07:37:53 UTC

Prime Density in Arithmetic Progressions and Seed Length of Nisan-Wigderson PRGs

Entry ID: ffe6d4da52cf **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 07:37:53 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Number Theory **Field B** (complexity object): Seed Length of Nisan-Wigderson PRGs

Statement:

For a 3-SAT instance with n variables, the seed length of a Nisan-Wigderson PRG that fools AC^0 circuits is $\Theta(\pi(n))$, where $\pi(n)$ is the number of primes $\leq n$. This holds for all instances where the clause graph is bipartite and the variable assignments induce a Dirichlet progression with modulus $\leq n^2$.

Rationale (proposer's reasoning):

The distribution of primes in arithmetic progressions relates to the structure of number fields, which could influence the pseudorandomness properties needed for PRGs. The density of primes affects the entropy required to generate pseudorandom bits, thus influencing seed length.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fa9739ba909bd214

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def sieve_of_eratosthenes(n):
    primes = [True] * (n + 1)
    p = 2
    while p * p <= n:
        if primes[p]:
            for i in range(p * p, n + 1, p):
                primes[i] = False
            p += 1
    return [p for p in range(2, n + 1) if primes[p]]

def is_prime(n):
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(math.sqrt(n)) + 1, 2):
        if n % i == 0:
            return False
    return True

def generate_dirichlet_progression(a, d, n):
    return [(a + k * d) % n for k in range(n)]

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random 3-SAT instance with n variables
    n = random.randint(5, 40)
    m = random.randint(2 * n, 4 * n)
    clauses = []
    for _ in range(m):
        literals = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        random.shuffle(literals)
```

```

    clauses.append(tuple(literals))

# Check if the clause graph is bipartite
variable_to_clauses = {i: [] for i in range(1, n + 1)}
for clause in clauses:
    for literal in clause:
        variable_to_clauses[abs(literal)].append(clause)

def bfs(start):
    queue = [start]
    visited = set()
    color = {start: 0}
    while queue:
        node = queue.pop(0)
        if node not in visited:
            visited.add(node)
            for neighbor in variable_to_clauses[node]:
                for lit in neighbor:
                    var = abs(lit)
                    if var != node and var not in visited:
                        if color.get(var) is None:
                            color[var] = 1 - color[node]
                            queue.append(var)
                        elif color[var] == color[node]:
                            return False
    return True

if not bfs(1):
    return {
        "metric_name": "Seed Length",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "Clause graph is not bipartite"
    }

# Compute  $\pi(n)$  using sieve of Eratosthenes
primes = sieve_of_eratosthenes(n)
pi_n = len(primes)

# Generate a Dirichlet progression based on clause graph bipartitioning
modulus = n * n
if not is_prime(modulus):
    return {
        "metric_name": "Seed Length",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "Modulus must be a prime number"
    }

progression = generate_dirichlet_progression(1, 2, modulus)

# Simulate PRG seed length via a pseudorandom generator with explicit modular arithmetic
seed_length = len(progression)

```


Instances tested is 0, violating the ‘n too small’ failure mode. The counterexample cites modulus requirements but no empirical data was collected to validate this. The conjecture’s metric (seed length) isn’t measured in any trials, making the ‘falsified’ verdict invalid without actual tests.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

No instances were tested, violating the ‘n too small’ failure mode. The counterexample references modulus requirements but lacks empirical validation. | next: Run tests with bipartite clause graphs and prime moduli to collect empirical data on seed length behavior

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	136116
2	propose	ollama_remote	qwen3:8b	0	0	101571
3	preregistration	ollama_remote	qwen3:8b	0	0	31631
4	novelty	ollama_remote	qwen3:8b	0	0	27222
5	novelty	ollama_remote	qwen3:8b	0	0	20400
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12139
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9980
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11706
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13843
10	critic	ollama_remote	qwen3:8b	0	0	32857
11	judge	ollama_remote	qwen3:8b	0	0	21453

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 418917 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/ffe6d4da52cf.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/ffe6d4da52cf.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/ffe6d4da52cf.tar.gz (if generated)